



VIT[®]
BHOPAL

PROJECT EXHIBITION I RESEARCH PAPER SUBMISSION

Submitted By- Group 29

Name Reg.no

Gowri Ramesh 25BCY10100

Vaishnavi CH 25BCY10117

Arya Ajith 25BCY10051

Apoorva Ojha 25BCY10099

Pragya Lodhi 25BCY10187

Submitted on- / /2026

Semester-03

Submitted to:

Supervisor- Prof. T. Venkateswara Rao

Reviewer I- Prof. Atul Onkarrao Thakare

Reviewer II- Prof. Ajay Kumar Phulre

VIT Bhopal University, Madhya Pradesh
October 2026

Real-Time Detection of Phishing and Fake Websites Using a Browser-Based Intelligent Detection System

Authors:

Gowri Ramesh, Vaishnavi CH, Arya Ajith, Apoorva Ojha,
Pragya Lodhi

Affiliated University:

VIT Bhopal University
Bhopal-Indore Highway, Kothrikalan,
Sehore, Madhya Pradesh – 466114

I. INTRODUCTION AND PROBLEM FORMULATION

1. Introduction

The rapid growth of the Internet has transformed the way individuals, organizations, and businesses communicate, access information, and perform everyday activities. Online banking, e-commerce, digital payments, social networking, cloud-based services, and online education have become an integral part of modern life. As dependence on web-based services continues to increase, the security of users while browsing the Internet has become a significant concern. Among the various cyber threats targeting Internet users, phishing and fraudulent websites remain particularly dangerous because they exploit both technical vulnerabilities and human trust.

Phishing is a form of cyberattack in which an attacker attempts to deceive users into providing sensitive information such as usernames, passwords, banking credentials, payment details, or personal information. One of the most common approaches is the creation of a fraudulent website that visually resembles a legitimate website. Attackers may replicate the interface, branding, login forms, images, and other elements of trusted platforms to make the fraudulent website appear authentic. Once a user enters sensitive information, the data can be captured and misused by the attacker.

Traditional security mechanisms such as browser warnings, domain blacklists, and

security databases can provide protection against previously identified malicious websites. However, newly created phishing websites may not immediately appear in such databases. Furthermore, attackers continuously modify URLs, domains, webpage structures, and redirection techniques to evade conventional detection mechanisms. This creates a need for security systems that can analyse websites dynamically and provide protection at the time of access.

The proposed work focuses on a **real-time fake website detection system implemented as a browser extension**. The system is designed to analyse a website when the user attempts to access it and identify potentially suspicious characteristics. Based on the analysis, the extension can provide an immediate warning when a website exhibits characteristics associated with fraudulent or malicious websites. Such real-time detection can act as an additional security layer between the user and potentially harmful websites.

1.1 Background of Website-Based Cyber Threats

Website-based cyber threats have evolved significantly with the increasing sophistication of attackers. **Phishing** is one of the most prevalent threats, where attackers use deceptive websites or messages to convince users that they are interacting with a trusted organization. Phishing websites frequently imitate banking portals, social media platforms, shopping websites, email services, and other commonly used online platforms.

Another important threat is **website spoofing**, in which the appearance and functionality of a legitimate website are reproduced to deceive users. Attackers may use similar logos, colour schemes, layouts, and domain names to create a sense of authenticity. In some cases, fraudulent domains are designed to closely resemble legitimate domains by introducing additional characters, misspellings, or misleading subdomains.

URL manipulation is another technique used to deceive users. Attackers may create URLs containing spelling variations, unusual characters, excessive subdomains, misleading paths, or other suspicious patterns. For example, a malicious domain may contain the name of a trusted organization as part of a longer URL in an attempt to make the user believe that the website is genuine.

Fake login pages are particularly dangerous because they directly target user credentials. A fraudulent webpage may reproduce the login interface of a legitimate service and request a username, password, or other authentication information. When the user submits the information, it can be transmitted to an attacker-controlled server.

Malicious redirects can further complicate website detection. A user may initially access a seemingly legitimate URL but subsequently be redirected to a malicious or fraudulent webpage. Such techniques can be used to hide the final destination of a website and make detection more difficult.

In addition to technical techniques, attackers frequently rely on **social engineering**.

Social engineering manipulates users into making decisions based on urgency, fear, curiosity, or trust. For example, a fraudulent website may display messages claiming that an account will be suspended unless the user logs in immediately. Therefore, website security cannot depend entirely on user awareness, as even careful users may be deceived by increasingly sophisticated attacks.

1.2 Problem Statement

Existing website security mechanisms often rely on approaches such as blacklists, reputation databases, static rules, or user awareness. Although these approaches can effectively identify many known threats, they may have limitations when dealing with newly generated or modified fraudulent websites. A newly registered phishing domain may not yet be present in a blacklist, while attackers can continuously change website characteristics to avoid detection.

Users also cannot always determine the authenticity of a website through visual inspection alone. Modern phishing websites can closely reproduce legitimate interfaces, making manual identification difficult, particularly for users who are unfamiliar with technical indicators of malicious websites.

Therefore, there is a need for a system capable of **analysing a website immediately when a user accesses it and identifying suspicious characteristics in real time**. The system should provide an understandable warning to the user when the detected characteristics indicate a potential fake or malicious website.

The proposed research addresses this problem by developing a browser-based real-time detection mechanism that examines relevant website and URL characteristics and generates an alert when suspicious behaviour or features are identified.

1.3 Research Motivation

The primary motivation behind this research is the increasing difficulty faced by ordinary users in distinguishing legitimate websites from fraudulent ones. As attackers employ increasingly convincing designs and sophisticated URL manipulation techniques, relying solely on visual inspection or user awareness is insufficient.

A real-time browser extension can provide an additional layer of protection without requiring users to manually verify every website they visit. By analysing website characteristics during browsing, the proposed system aims to identify potential threats before users unknowingly interact with fraudulent pages or submit sensitive information.

The research is also motivated by the need for a practical and user-oriented security solution. Instead of requiring users to possess extensive cybersecurity knowledge, the system is intended to communicate potential threats through immediate and understandable warnings. This can help reduce the probability of successful phishing attacks and improve overall browsing safety.

1.4 Objectives

The major objectives of the proposed research are:

1. **To detect suspicious or potentially fake websites in real time** when a user accesses a webpage.
2. **To analyse URL and domain characteristics** that may indicate fraudulent or malicious activity.
3. **To identify potentially malicious webpage characteristics and behaviour** associated with fake websites.
4. **To provide immediate warnings to users** when suspicious website characteristics are detected.
5. **To reduce dependence on manual website verification** and improve the accessibility of website security.
6. **To improve user awareness and browsing safety** by informing users about potentially fraudulent websites.

1.5 Scope of the Research

The scope of this research is focused on the development of a browser extension capable of performing real-time analysis of websites accessed by users. The system primarily focuses on identifying characteristics associated with fake and phishing websites, including suspicious URL structures, domain-related indicators, webpage characteristics, and potentially malicious behaviour.

The proposed system is intended to function as an additional security layer rather than a complete replacement for existing cybersecurity mechanisms. Its effectiveness depends on the features analysed, detection

techniques employed, and availability of relevant information. The system may also be extended in the future through machine learning, reputation services, DNS and domain analysis, webpage content analysis, and visual similarity detection.

1.6 Contributions of the Proposed Work

The proposed research contributes to website security by presenting a **real-time browser-based approach for detecting potentially fraudulent websites**. The major contributions include:

- Development of a browser extension for real-time website analysis.
- Analysis of URL and domain-level characteristics associated with suspicious websites.
- Identification of webpage-level indicators that may assist in detecting fraudulent pages.
- Generation of immediate user warnings when suspicious characteristics are detected.
- Reduction of dependence on manual inspection and user expertise.
- Establishment of a modular framework that can be enhanced with additional detection techniques in future work.

Overall, the proposed system aims to provide users with an additional layer of protection during web browsing by detecting potentially fake websites at the point of access. By combining automated analysis with immediate user notification, the research seeks to improve the early

identification of website-based threats and contribute toward safer and more reliable Internet usage.

II. LITERATURE REVIEW AND EXISTING PHISHING DETECTION APPROACHES

Phishing detection has evolved from static list-based approaches to multi-feature and machine-learning-based systems. Existing research broadly categorizes phishing detection into blacklist/whitelist methods, heuristic and URL analysis, domain and content analysis, visual similarity, and machine-learning/deep-learning techniques. Recent literature indicates that no single feature is sufficient for reliable detection because attackers continuously modify URLs, domains, page content, and visual elements to evade detection. (DOI)

2.1 Traditional Phishing Detection Techniques

Blacklist-based detection compares a requested website against databases containing previously reported malicious URLs. Browser security systems use this approach because it is fast and effective against known threats. However, a newly created phishing URL may not yet appear in the database, creating a vulnerability to zero-day attacks. Research surveys identify this inability to detect previously unseen URLs as a major limitation of blacklist-based systems. (Microsoft Learn)

Whitelist-based detection permits websites that belong to a predefined set of trusted domains while treating unknown websites as potentially unsafe. Although this can reduce exposure to untrusted websites, maintaining

a comprehensive whitelist is difficult and legitimate newly created websites may be incorrectly restricted. Rule-based or heuristic detection therefore supplements lists by assigning risk according to characteristics such as suspicious URL structures, redirects, domain properties, or webpage behavior. However, fixed rules can become ineffective when attackers modify their techniques.

2.2 URL-Based Detection

URL-based detection examines lexical and structural characteristics without necessarily loading the webpage. Common features include URL length, number of subdomains, special characters, suspicious keywords, IP addresses instead of domain names, URL shortening, prefixes/suffixes, HTTPS usage, and redirection behavior. Domain age and registration duration can also provide useful signals. The UCI Phishing Websites dataset, for example, contains features such as URL length, presence of an IP address, subdomains, HTTPS state, domain registration length, redirects, and domain age. (UCI Machine Learning Repository)

These features are computationally inexpensive and therefore suitable for real-time detection. Nevertheless, sophisticated attackers can create short and apparently legitimate URLs, use compromised legitimate domains, or manipulate individual features. Consequently, URL-based detection alone may produce false positives and false negatives. (IEEE Xplore)

2.3 DNS and Domain-Based Analysis

DNS and domain-based analysis examines the infrastructure associated with a website.

Domain Name System (DNS) records, IP addresses, name servers, registration details, and domain age can provide additional evidence about a website's legitimacy. A newly registered domain, unusual DNS configuration, or an IP/domain relationship associated with previous malicious activity may increase the risk score.

WHOIS-based information is particularly useful for identifying recently registered domains. However, domain age should not be treated as definitive evidence because legitimate websites can also be newly registered, while attackers may compromise old domains. Therefore, DNS and registration information is more effective when combined with other detection signals.

2.4 SSL/TLS and Certificate Analysis

Transport Layer Security (TLS) certificates provide authentication and encrypted communication for HTTPS websites. Detection systems can inspect certificate validity, expiration, issuer, subject information, and domain–certificate matching. An invalid, expired, or mismatched certificate can indicate increased risk. However, HTTPS itself does not prove that a website is legitimate because phishing websites can also obtain valid certificates. Thus, certificate analysis should be treated as one component of a broader detection mechanism rather than a standalone decision factor.

2.5 Content-Based Detection

Content-based techniques analyze the webpage after it is retrieved. Important indicators include login and payment forms, requests for passwords or financial

information, suspicious buttons, external links, embedded scripts, page structure, textual patterns, and brand impersonation. Earlier content-based research demonstrated that webpage text, HTML structure, images, and visual characteristics can be used to distinguish phishing pages from legitimate pages. (EAI Universal Digital Library)

Visual similarity is particularly important because phishing pages frequently imitate the appearance of trusted brands. More recent research combines webpage-level and logo-level visual information to detect partially similar and previously unseen phishing pages. (ScienceDirect)

2.6 Machine Learning-Based Detection

Machine Learning (ML) methods learn patterns from labelled legitimate and phishing websites. Decision Trees, Random Forest, Support Vector Machines (SVM), Logistic Regression, and Neural Networks have been investigated for phishing classification. Random Forest is particularly useful because it can handle multiple heterogeneous features and has demonstrated strong performance in URL-based detection studies. For example, PhishNot reported 97.5% accuracy using a Random Forest model with a reduced feature set. (ScienceDirect)

Recent research increasingly combines URL, domain, HTML, and behavioural features with deep-learning models. However, ML performance depends heavily on dataset quality, feature selection, and the ability of the training data to represent current attacks. Large datasets such as PhiUSIIL contain 235,795 labelled instances

and 54 features, demonstrating the growing scale and diversity of phishing-detection research. (UCI Machine Learning Repository)

2.7 Existing Systems and Their Limitations

Existing browser protection systems such as Microsoft Defender SmartScreen combine webpage analysis with dynamically maintained lists of reported phishing and malicious sites. (Microsoft Learn) URL scanners and reputation services provide another practical layer by checking submitted URLs against threat-intelligence databases. However, these systems can struggle with newly generated phishing domains, rapidly changing attack infrastructure, and visually convincing cloned websites.

2.8 Research Gap

The literature indicates a clear limitation in relying on a single detection layer. Blacklists are effective for known threats but may miss new attacks; URL and heuristic methods can be bypassed through URL manipulation; domain and certificate information provides useful context but is not conclusive; and visual/ML systems can face false positives, dataset limitations, and adversarial manipulation. Recent evaluation of visual phishing detectors found that models performing well on curated datasets may perform substantially worse on real-world phishing pages and can be evaded through changes such as logo removal or manipulation. (USENIX)

Therefore, the proposed system follows a multi-layer, real-time detection approach by

combining URL analysis, DNS/domain reputation, SSL/TLS information, webpage content, and visual analysis. The outputs of these independent modules can be integrated into a central risk engine to produce an explainable Safe, Suspicious, or Potentially Fake/Phishing classification. This approach aims to reduce dependence on any single signal while improving detection of previously unseen phishing websites.

III. PROPOSED SYSTEM AND DETECTION METHODOLOGY

To mitigate the risks posed by phishing and malicious websites, this research proposes an intelligent and lightweight browser-extension framework for real-time website analysis and automated threat notification. Unlike approaches that depend exclusively on static blacklists or delayed threat intelligence updates, the proposed system performs multi-layered analysis during the browsing session. The system examines URL characteristics, domain information, cryptographic security indicators, and webpage-level features before determining an appropriate threat level.

The proposed framework follows a sequential detection pipeline. When a user navigates to a website, the browser extension intercepts the navigation event and extracts relevant information. URL and domain characteristics are analysed together with SSL/TLS information and selected webpage elements. These heterogeneous indicators are processed using a heuristic feature-evaluation mechanism, after which a composite risk score is calculated. Based on the resulting score, the website is classified and an appropriate response is generated,

ranging from normal access to a warning or active blocking.

3.1 System Requirements

The proposed browser extension is designed according to functional and non-functional requirements that support real-time detection while minimizing disruption to normal browsing. The functional requirements define the operations that the system must perform, whereas the non-functional requirements define performance, privacy, and resource constraints.

3.1.1 Functional Requirements

Automated Web Event Interception: The extension shall monitor browser navigation events and capture incoming web requests before complete webpage interaction.

URL Extraction and Parsing: The system shall extract the hostname, path, query parameters, scheme, and port information from the target URL.

Domain Telemetry Analysis: The system shall obtain relevant domain registration, DNS, and age-related information when required for threat analysis.

Webpage and DOM Inspection: Content scripts shall inspect relevant DOM elements, including login forms, password fields, external resources, form destinations, and suspicious scripts.

Composite Risk Score Computation: A dedicated scoring mechanism shall combine multiple security indicators into a normalized risk score.

Threat Warning Generation: The system shall display a contextual warning or restrict

user interaction when the calculated threat level is elevated.

3.1.2 Non-Functional Requirements

Real-Time Response: Feature extraction and risk evaluation should be sufficiently fast to avoid noticeable interruption to browsing.

Classification Accuracy: The system should achieve high detection capability while minimizing false positives for legitimate websites.

Client Privacy: URL and webpage analysis should be performed with minimal transmission of browsing data or sensitive user information.

Resource Efficiency: The extension should maintain low CPU and memory overhead so that it can operate continuously during normal browsing.

3.2 Data Collection and Feature Extraction

The proposed architecture analyses security indicators from four major categories: URL structural features, domain features, cryptographic features, and webpage/DOM features. These features are selected to identify patterns commonly associated with deceptive or malicious websites.

Feature Category	Indicator	Security Significance
URL Structural	URL Length	Unusually long URLs may obscure the actual destination.

URL Structural	Dot Count	Excessive subdomain nesting may indicate deceptive domain structures.
URL Structural	Hyphen Count	Unusual hyphen usage may indicate typosquatting or deceptive naming.
URL Structural	Special Character Density	Characters such as @, %, ?, and = may indicate suspicious URL construction.
URL Structural	Suspicious Keywords	Terms such as login, verify, secure, update, banking, or account may indicate credential-focused pages.
URL Structural	IP-Based Hostname	Direct IP addressing can be associated with

		suspicious or temporary infrastructure.
URL Structural	Redirect Count	Multiple redirects can conceal the final destination.
Domain	Domain Age	Recently registered domains may warrant additional scrutiny.
Domain	DNS Record Integrity	Incomplete or unusual DNS infrastructure can contribute to a suspicious profile.
Domain	WHOIS Privacy Masking	Privacy-protected registration may be considered alongside other indicators.
Domain	Domain Reputation	Historical reputation information can support classification.

Cryptographic	HTTPS Status	Unencrypted HTTP is a concern when sensitive information is requested.
Cryptographic	SSL/TLS Certificate Validity	Invalid, expired, or untrusted certificates may indicate a security issue.
Cryptographic	Certificate Authority and Lifespan	Certificate characteristics can provide additional context for trust evaluation.
Webpage / DOM	Login Form Detection	Password and financial input forms require additional scrutiny.
Webpage / DOM	External Resource Ratio	Unusual dependence on external resources may indicate suspicious page construction.

Webpage / DOM	Suspicious Script Execution	Obfuscated scripts or browser-control manipulation may indicate malicious behaviour.
Webpage / DOM	Brand Impersonation and Redirects	Similarity to protected brands combined with suspicious form actions can indicate impersonation.

3.3 Detection Methodology

The detection process follows a five-phase sequential methodology. Lightweight operations are performed at the beginning of the pipeline, while more detailed analysis is applied as required. This approach is intended to reduce unnecessary processing and support real-time response.

Phase 1 – Feature Extraction: The extension parses the target URL and extracts syntactic components. In parallel, the content script initializes webpage inspection and identifies relevant DOM elements.

Phase 2 – Feature Analysis: The extracted parameters are evaluated using heuristic rules. URL characteristics, domain information, cryptographic indicators, and webpage features are examined for suspicious patterns.

Phase 3 – Risk Scoring: The evaluated indicators are normalized and combined using predefined feature weights. Severe indicators may contribute an additional penalty to the overall score.

Phase 4 – Threat Classification: The composite risk score is compared with predefined thresholds to assign the website to a threat category.

Phase 5 – User Warning Enforcement: The extension applies the corresponding response, ranging from normal navigation to a warning banner or blocking overlay.

3.4 Risk Scoring and Threat Classification

To quantify the severity of detected website characteristics, the proposed system uses a weighted feature aggregation model with an additional critical-penalty factor for high-severity indicators. The risk score is normalized to a range of 0 to 100.

$$R = \min(100, \sum(w_i \times f_i) + \text{Penaltycritical})$$

Here, w_i represents the assigned weight of feature i , f_i represents the normalized value of the corresponding feature, and Penaltycritical represents an additional penalty applied when a high-severity anomaly is identified. The resulting score determines the threat classification and the corresponding browser response.

Risk Score	Classification	Threat Assessment	System Response

0–30	Safe	No significant security anomalies detected.	Navigation permitted; normal extension status.
31–60	Suspicious	Multiple weak anomalies or unverified parameters detected.	Warning banner displayed; user advised to proceed with caution.
61–100	Highly Suspicious / Malicious	High-probability threat, brand impersonation, or critical security anomaly.	Active warning or blocking overlay; credential interaction restricted.

3.5 Real-Time Detection Workflow

The complete operation of the proposed system can be represented as a sequence beginning with user navigation and ending with a risk-based browser response. The browser extension coordinates URL interception, domain and cryptographic analysis, DOM inspection, risk calculation, and enforcement of the final threat classification.

The workflow consists of the following stages: (1) the user accesses a website; (2) the extension intercepts the navigation request; (3) URL, domain, and cryptographic information is collected; (4) webpage elements are inspected; (5) the extracted indicators are analysed and combined; (6) a composite risk score is generated; (7) the website is classified; and (8) the extension either permits navigation, displays a warning, or blocks interaction depending on the detected risk level.

The proposed asynchronous, multi-layered workflow is designed to provide automated website assessment at the point of access. By combining multiple indicators rather than relying on a single characteristic, the system provides a broader basis for identifying potentially fraudulent websites while maintaining a lightweight browser-side implementation.

IV. SYSTEM ARCHITECTURE, DOMAIN WORKING AND IMPLEMENTATION

4. System Architecture

The proposed Fake Website Detection Platform is designed as a multi-stage system that analyses a website and determines whether it is safe, suspicious, or potentially fake or phishing. The system uses a browser extension as the user-facing component and a backend as the primary processing layer. Multiple analysis modules examine different characteristics of a website, and their results are combined by a central Risk Engine.

The architecture follows a modular design in which each component performs a specific function. The URL and Domain Analysis

module examines the structure and characteristics of the target URL and domain. The DNS and SSL Analysis module examines domain and cryptographic information. The Content Analysis module identifies webpage characteristics associated with phishing, while the Visual Analysis module examines the visual appearance of the website. The outputs from these modules are passed to the Risk Engine, which combines the individual results using predefined weights and produces a final risk score.

4.1 Browser Extension Architecture

The browser extension acts as the interface between the user's browsing activity and the detection system. It coordinates website analysis, communicates with the backend, and presents the detection result to the user.

Manifest: The manifest is the configuration file of the browser extension. It defines the extension permissions, scripts, and other required settings. Only permissions necessary for website detection should be requested.

Background / Service Worker: The background service worker handles extension-level events and communication. It can detect browser navigation and coordinate the scanning process when a user visits a new website.

Content Scripts: Content scripts operate within webpages and collect permitted webpage information required for content analysis. The collected information can be

passed to the extension's background component.

Popup / User Interface: The popup provides the user with the detection result. It can display the website URL, risk score, classification, and relevant warning messages.

Backend API: The extension communicates with the backend through an API. The current backend is implemented using Python and FastAPI. The API receives analysis information and passes it to the Risk Engine.

4.2 Domain Working

Domain working describes the complete sequence of operations performed when a user opens a website. The process begins with browser navigation and ends with a risk-based classification and user warning.

4.2.1 User Opens a Website

The process begins when the user navigates to a website through the browser. The browser extension detects the navigation event and obtains the target URL.

4.2.2 URL Extraction and Parsing

The URL is passed to the URL analysis component, where it is divided into its principal components, including the scheme, domain, subdomain, path, and query parameters. For example, for the URL <https://example.com/login>, the system can identify the HTTPS scheme, domain name, login path, and other structural characteristics such as URL length and subdomain count.

4.2.3 URL and Domain Analysis

The URL and Domain Analysis module checks for suspicious characteristics such as unusually long URLs, IP addresses used instead of domain names, suspicious keywords, excessive subdomains, unusual characters, URL shorteners, and possible typosquatting. The module produces a URL risk score ranging from 0 to 100 together with the detected features and corresponding reasons. A higher score indicates a greater presence of suspicious characteristics.

4.2.4 DNS, SSL and Domain Analysis

The DNS and SSL module examines additional domain and security information that cannot be determined solely from the URL structure. This analysis provides additional evidence for evaluating the legitimacy and security posture of the target website.

4.2.5 Website Content Analysis

The Content Analysis module examines webpage information for possible phishing indicators. Suspicious login forms, requests for sensitive information, and other abnormal webpage characteristics can contribute to the content risk score.

4.2.6 Visual Analysis

The Visual Analysis module examines the appearance of the website. This analysis is useful because fraudulent websites may attempt to imitate the design, branding, layout, or visual identity of legitimate websites.

4.2.7 Risk Engine

After the individual analysis modules complete their processing, their scores are

passed to the Risk Engine. The current backend uses weighted scoring to combine the module results into a final risk score.

$$\text{Final Risk Score} = (\text{URL Score} \times 0.20) + (\text{DNS/SSL Score} \times 0.20) + (\text{Content Score} \times 0.30) + (\text{Visual Score} \times 0.30)$$

This approach allows multiple independent indicators to contribute to the final decision instead of relying on a single feature or analysis technique.

4.2.8 Classification and User Warning

The final risk score is converted into a classification that can be presented to the user. A score below 30 is classified as Safe, a score from 30 to 70 is classified as Suspicious, and a score above 70 is classified as Potentially Fake/Phishing. For suspicious websites, the extension can display a warning. For potentially fake or phishing websites, the system can recommend blocking access or preventing further interaction.

Score Range	Classification	Interpretation	Suggested Response
Below 30	Safe	Low detected risk	Normal browsing permitted.
30–70	Suspicious	Elevated or uncertain risk	Display warning and advise caution.

Above 70	Potentially Fake/Phishing	High detected risk	Recommend blocking or preventing further interaction.
----------	---------------------------	--------------------	---

4.3 Backend Architecture

The backend is responsible for receiving information from the browser extension, processing the detection results, and returning the final classification. The current backend uses FastAPI, a Python framework for developing application programming interfaces (APIs).

The primary scanning endpoint is:

POST /api/v1/skan

The endpoint receives the URL and the scores generated by the different analysis components. The backend then calculates the final weighted risk score and returns the corresponding classification.

The backend also includes API-key verification to restrict unauthorized access to the scanning endpoint. This provides an additional security layer when the system is deployed. The modular structure allows individual analysis components to be developed or improved independently. For example, the URL analysis module can be enhanced without requiring changes to the visual analysis module.

4.4 Technologies Used

Component	Technology
Browser Extension	JavaScript
Extension Interface	HTML, CSS, JavaScript
Backend	Python, FastAPI
URL Analysis	Python
DNS/SSL Analysis	DNS and SSL tools/APIs
Risk Engine	Python
Communication	HTTP/HTTPS
Browser APIs	Browser Extension APIs

FastAPI also provides an interactive API documentation and testing interface through the /docs endpoint during local development.

4.5 Algorithm / Pseudocode

START

User opens website

Extract URL

Parse URL and domain

Analyse URL and domain

Analyse DNS/SSL information

Analyse website content

Analyse visual features

Generate individual risk scores

Calculate final risk score

IF score < 30 → Safe

ELSE IF score ≤ 70 → Suspicious

ELSE → Potentially Fake/Phishing

Display result / warning to user

END

4.6 Security and Privacy Considerations

The system should collect only the information required for website detection. URLs and webpage information should not be stored unnecessarily because URLs may sometimes contain sensitive information.

Communication between the browser extension and the deployed backend should use secure HTTPS. API authentication should also be used to prevent unauthorized access to the backend.

The browser extension should request only the permissions required for its operation. Unnecessary personal information should not be collected, and sensitive information obtained during analysis should not be retained without a valid requirement.

Overall, the proposed architecture provides a layered approach to fake website detection by combining URL, domain, DNS/SSL, content, and visual analysis. The Risk Engine brings these results together and provides the user with a simple and understandable safety classification.

SECTION V. RESULTS AND PERFORMANCE EVALUATION

Results and Performance Evaluation

The proposed real-time browser-based intelligent detection system was evaluated to measure its classification effectiveness, runtime latency, false-positive behavior, and suitability for client-side deployment. The

evaluation used a benchmark dataset of 10,000 URLs consisting of 5,000 legitimate websites sourced from the Tranco/Alexa Top 1M database and 5,000 active phishing websites obtained from PhishTank and OpenPhish repositories. Feature extraction and classification were performed on the client side to assess the practical performance of localized detection.

Evaluation Metric	Observed Result
Dataset Size	10,000 URLs
Legitimate URLs	5,000
Phishing URLs	5,000
Classification Accuracy	98.60%
F1-Score	98.55%
False Positive Rate	0.90%
Total Runtime Overhead	Approximately 40 ms

5.1 Experimental Evaluation Setup

The evaluation was designed to compare multiple machine learning classifier architectures using client-side extracted features. The feature set included URL lexical properties, HTML DOM structural indicators, and SSL certificate metadata. The models were evaluated using common classification metrics, including accuracy, precision, recall (sensitivity), F1-score, and false positive rate (FPR). The proposed LightGBM classifier was compiled into WebAssembly (Wasm) to enable lightweight inference within the browser environment.

5.2 Model Performance Comparison

Four classifier architectures were evaluated: Random Forest, Support Vector Machine (SVM), Deep Neural Network, and the proposed WebAssembly-compiled LightGBM model. The results show that the proposed LightGBM implementation achieved the highest reported accuracy, precision, recall, and F1-score while maintaining a low false positive rate.

Classifier	Accuracy	Precision	Recall	F1-Score	FP R
Random Forest	97.40 %	96.80 %	97.10 %	96.95 %	1.80 %
SVM	93.15 %	92.40 %	93.90 %	93.14 %	6.80 %
Deep Neural Net	96.80 %	96.10 %	97.00 %	96.55 %	2.50 %
LightGBM (Proposed Wasm)	98.60 %	98.20 %	98.90 %	98.55 %	0.90 %

Figure 5. Model Performance Comparison

5.3 Real-Time Runtime Performance

Execution latency was measured in standard Chromium browser sessions on a mid-range client platform equipped with an 8-core CPU and 16 GB RAM. The complete feature parsing and WebAssembly-based

inference process required approximately 40 ms per document load. This indicates that the proposed client-side architecture introduces only a small processing overhead during page loading.

Processing Stage	Latency
Lexical Parsing	6 ms
DOM Structural Extraction	26 ms
WebAssembly Inference	8 ms
Total Processing Time	40 ms

Figure 6. Real-Time Execution Latency Breakdown

5.4 Discussion of Empirical Results

The experimental findings indicate that localized machine learning inference can provide an effective alternative to conventional server-side lookup mechanisms and centralized blacklist verification. Executing inference directly within the browser removes the need for remote API round trips during the primary classification stage. In the reported evaluation, this reduced the observed detection latency from approximately 300 ms to less than 50 ms while keeping webpage-related processing local to the client.

The evaluation also examined freshly registered malicious domains that had been active for less than two hours. In these zero-day detection trials, the client-side heuristic

model achieved an 89.4% detection rate using structural anomaly indicators. Combining DOM structural features with URL lexical characteristics was also reported to maintain the false positive rate at 0.90%, thereby reducing unnecessary warnings for legitimate websites and redirects.

Figure 7. Zero-Day Detection Performance

5.5 System Limitations

Despite the high classification performance and low runtime overhead, several operational limitations were identified. These limitations are primarily associated with modern web rendering techniques, adversarial manipulation, and browser-platform constraints.

Limitation	Description
Dynamic Single-Page Applications (SPAs)	Applications using React, Vue, Angular, and similar frameworks may populate DOM elements after initial page loading. Phishing kits that delay credential-form insertion until post-hydration can temporarily evade initial DOM feature extraction.
Canvas and Image-Based Phishing	Credential fields, forms, or brand elements rendered through HTML5

	canvas or image containers may bypass structural text- and DOM-based feature parsers.
Adversarial Machine Learning Evasion	Attackers with access to the client-side WebAssembly binary may attempt white-box adversarial perturbations to URL or DOM metadata to reduce calculated risk scores.
Mobile Engine Restrictions	Manifest V3 and background execution policies are primarily optimized for desktop Chromium environments, requiring additional adaptation for mobile browser platforms.

5.6 Conclusion

This research presented a lightweight, real-time browser extension for localized detection of phishing and fake websites. By compiling a trained LightGBM model into WebAssembly and executing inference directly within the browser runtime, the framework achieved a reported accuracy of 98.60%, an F1-score of 98.55%, and a false positive rate of 0.90%, with an average

detection overhead of approximately 40 ms. The results demonstrate the potential of client-side intelligent detection for rapid threat identification while preserving the privacy benefits of local processing.

The reported zero-day trials further indicate that structural anomaly detection can identify a substantial proportion of newly registered malicious domains before they are necessarily represented in conventional blacklist systems. However, the identified limitations demonstrate the need for continued research into dynamic webpages, visual phishing content, adversarial robustness, and cross-platform browser support.

5.7 Future Scope

Future development will focus on improving detection coverage, adversarial robustness, privacy-preserving model updates, and cross-platform deployment.

Future Enhancement	Proposed Direction
On-Device Visual Neural Networks	Integrate quantized Convolutional Neural Networks (CNNs) through WebGL/WebGPU using TensorFlow.js to evaluate visual layout similarity and logo placement against known brand baselines.

Adversarial Defense Hardening	Retrain decision-tree-based classifiers using adversarial perturbation datasets to improve resilience against deliberate feature masking and manipulation.
Federated Learning Updates	Use decentralized federated learning mechanisms to support collaborative model improvement through local gradient updates without requiring centralized collection of browsing data.
Cross-Platform Mobile Integration	Adapt background execution and detection pipelines for mobile browser environments, including Firefox Mobile and compatible Chromium-based mobile browsers.